🏥 Project Proposal : ICD-10 Code Blue

A Diagnostic Simulation & Data Analytics Game

1. Project Overview

ICD-10 Code Blue is a medical simulation game designed to bridge the gap between healthcare informatics and gamified learning. Players act as medical professionals who must diagnose patients by matching clinical symptoms with the correct **ICD-10 codes** under high-pressure conditions. The project aims to provide an educational tool that tracks and visualizes the user's learning efficiency through data science.

---

2. Project Review

Traditional medical quiz apps or basic ICD-10 search directories.

Specific Improvements x Gamification

Unlike static directories, this project introduces a Patient Stability mechanic—if the

player takes too long or enters the wrong code, the patient's vitals drop, leading to a ***Code Blue*** (Game Over).

- **Performance Analytics**
  Most existing apps don't track learning progress. This project includes a built-in statistical engine that monitors which disease categories the player struggles with, providing a visual *Learning Curve* after gameplay.

---

3. Programming Development

3.1 Game Concept

The game presents a series of patient cases. Each case includes a text description of symptoms (e.g., "Sharp chest pain radiating to the left arm"). The player must use a search interface to identify the specific ICD-10 code (e.g., I21.9).

Core Loop
Patient presents symptoms -> Player searches ICD-10 database -> Code entered -> Stability update.

- **Key Features:** Randomly generated patient scenarios, a countdown timer, a scoring system based on diagnostic precision, and a *Medical Handbook* UI for code lookups.

3.2 Object-Oriented Programming Implementation

1. **Patient**
   Represents a patient case in the game, including symptoms and the correct ICD-10 diagnosis used for gameplay.

2. **ICD_Database**
   Attributes:
   - code_list
   - description_list

   Methods:

   - load_data(file_path)
   - search_code(keyword)
   - filter_by_category(category)

3. **GameController**
   The main engine that manages the game state (Menu, Playing, Game Over), score calculation, and time limits.

   Attributes:

   - game_state
   - score
   - time_remaining

   Methods:

   - start_game()
   - update_game_state()
   - calculate_score()
   - end_game()

4. **StatsTracker**
   A dedicated class responsible for logging every player action (clicks, keys, time) into a data file.

   Attributes:

   - data_records

   Methods:

   - log_attempt(data)
   - save_to_csv()

5. **UI_Manager**
   Handles the graphical interface (using Tkinter or Pygame), rendering buttons, text, and the health bar.

   Attributes:

   - window
   - UI_elements

   Methods:

   - render_screen()
   - update_health_bar()
   - display_text()

3.3 Algorithms Involved

1. Randomized Sampling

To ensure gameplay variety, the system selects patient cases randomly from the database without immediate repetition.

Process:

- Load all patient cases into a list
- Shuffle the list using a randomization function
- Sequentially present cases to the player
- Reset and reshuffle when all cases are used

2. Difficulty Scaling

The game dynamically adjusts difficulty based on player performance.

Logic:

- If the player score increases → reduce available time
- If the player makes repeated errors → maintain or slightly increase time

Example Rule:

```
if score >= 10:

  time_limit -= 2 seconds
```

3. Search Algorithm (Fuzzy Matching)

The system implements a fuzzy matching algorithm to improve search usability.

Instead of exact matching, the system compares user input with ICD-10 entries and returns the closest matches.

Process:

- Receive user input string
- Compare with all ICD-10 codes/descriptions
- Calculate similarity score (e.g., Levenshtein Distance)
- Return top matching results

---

4. Statistical Data (Prop Stats)

4.1 Data Features

The game will collect 100+ records for each of the following 5 features:

1. **Response Time (seconds)**
   Time from patient appearance to correct diagnosis

2. **Keystroke Efficiency**
   Number of keystrokes compared to the correct code length

3. **Accuracy Rate**
   Percentage of correct vs. incorrect code entries per session.

4. **Category Frequency**
   Tracks the ICD-10 category for each individual diagnosis attempt to ensure sufficient data points (100+ records)

5. **Stress Factor**
   Correlation between the remaining time on the clock and the error rate.

Table statistic data

Feature	Why it is good / What it is used for	How will you obtain 100 values?	Which variable (class)?	How to display
Response Time	Measure player speed and learning improvement	Record every attempt during gameplay (100+ attempts)	`response_time` from StatsTracker	Line graph (trend over attempts), Mean & SD
Keystroke Efficiency	Evaluate how efficiently players input codes	Log keystrokes for each attempt	`keystrokes` from StatsTracker	Box plot or Histogram
Accuracy Rate	Measure correctness of diagnosis	Track correct vs incorrect per attempt	`accuracy` from StatsTracker	Pie chart or Bar chart
Category Frequency	Identify which ICD-10 categories are most used	Record category for each diagnosis attempt	`category` from StatsTracker	Bar chart
Stress Factor	Analyze impact of time pressure on errors	Record remaining time + error per attempt	`remaining_time` + `accuracy`	Scatter plot (correlation)

Table statistic value

	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	Response Time Improvement	Show how user speed improves over time	Line Graph	Attempt Number	Response Time (seconds)
Graph 2	Error Distribution	Show frequency of incorrect diagnoses	Histogram	Number of Errors per Attempt	Frequency
Graph 3	Category Frequency	Show how often each ICD-10 category appears	Bar Chart	ICD-10 Categories	Number of Occurrences
Graph 4	Accuracy Rate	Show proportion of correct vs incorrect answers	Pie Chart	Result Type (Correct/Incorrect)	Percentage (%)
Graph 5	Response Time Distribution	Show spread of response times	Box Plot	Attempts	Response Time (seconds)

4.2 Data Recording Method

All gameplay data will be stored in a single global CSV file and appended in real time. Each row represents one diagnostic attempt.

CSV Structure:

- timestamp
- response_time
- keystrokes
- accuracy
- category
- remaining_time

This structure ensures that all features meet the 100-record requirement during testing.

4.3 Data Analysis Report

Measures:

- Mean and Standard Deviation of response time
- Accuracy trends across multiple attempts

Visualization:

- **Histogram of Error Distribution**
  The histogram represents the frequency of incorrect diagnoses.
- X-axis: Number of errors per attempt (from *Accuracy Rate*)
- Y-axis: Frequency of occurrences
  This helps identify how often players make mistakes and detect patterns in error distribution.
- **Line Graph of Response Time Improvement**
  The line graph shows how the player's response time changes over multiple attempts.
- X-axis: Attempt number (sequence of gameplay records)

- Y-axis: Response Time (seconds)
  This visualization demonstrates learning progress by showing whether the player becomes faster over time.
- **Category Distribution Chart (Bar Chart)**
  This chart represents how frequently each ICD-10 category appears.
- X-axis: ICD-10 categories (from *Category Frequency*)
- Y-axis: Number of occurrences
  This helps analyze which medical categories are most common and where players may need more practice.

Feature	Measure	Description
Response Time	Mean	Average time taken to diagnose a patient
Response Time	Median	Middle value of response time across attempts
Response Time	Standard Deviation	Measures variation in response time
Accuracy Rate	Mean	Average percentage of correct diagnoses
Accuracy Rate	Mode	Most frequent accuracy outcome (e.g., correct or incorrect)
Keystroke Efficiency	Mean	Average number of keystrokes per attempt
Remaining Time	Mean	Average remaining time after each attempt

5. Project Planning Timeline

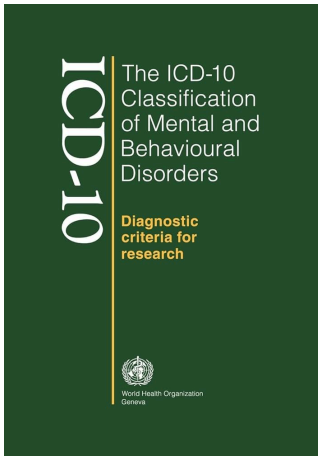
Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	**Database & Logic:** Clean ICD-10 dataset and build the Patient class.
3	10	**Core Gameplay:** Develop the GameController and search mechanics.
4	11	**GUI Design:** Implement the visual interface.
5	12	**Stats Integration:** Code the StatsTracker to log CSV data.
6	13	**Data Collection & Viz:** Playtest to generate 100+ records and code the charts.
7	14	Submission week (Draft)

6. Document version

Version: 1.0
Date: 13 March 2026
Editor: Danita Frikaow

References

World Health Organization. (1993). *The ICD-10 classification of mental and behavioural disorders: Diagnostic criteria for research*. World Health Organization.



Date	Name	Feedback
17/03	Peraya	**Major Changes Needed** - **Section 3.2:** You listed class names, but the requirement specifically asks for key attributes and methods. Without these, the OOP implementation is just a list of names, not a design. - **3.3 Algorithm Commitment:** You cannot use “optional” or “maybe” in a proposal. You must definitively state which **Search Algorithm** you will implement. - **4.1 Statistical Feature Design:** "Category Frequency" is poorly defined. If you only count categories per game session, you will fail the **100 records per feature** requirement. You must redefine this to track the category of *every individual diagnosis attempt* to ensure you hit the 100+ data point threshold. - **4.3 Analysis Depth:** Simply calculating a Mean is insufficient for useful visualization. You must explain how the visualization specifically helps the player or developer. **Minor Changes Needed** - **4.2 Data Storage:** Specify the **structure** of your CSV. Will it be one global file or one per session? Clearer file handling logic is needed to ensure data isn't overwritten.